

## SSCD LAB ASSIGNMENT -3

INPUT CODE-

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
using namespace std;

struct MNTEntry {
    int index;
    string macroName;
    int mdtIndex;
};

struct ALAEntry {
    int index;
    string dummyArgument;
};

int main() {

    string fileName;

    cout << "Enter input file name: ";
    cin >> fileName;

    ifstream input(fileName);

    if (!input) {
        cout << "File cannot be opened!" << endl;
        return 1;
    }

    ofstream intermediate("intermediate.txt");
    ofstream mdtFile("mdt.txt");
    ofstream mntFile("mnt.txt");
    ofstream alaFile("ala.txt");

    vector<string> MDT;
    vector<MNTEntry> MNT;
    vector<ALAEntry> ALA;
```

```

string line;
bool insideMacro = false;
string macroName = "";

int mntIndex = 1;
int mdtIndex = 1;
int alaIndex = 1;

while (getline(input, line)) {

    if (line.empty())
        continue;

    stringstream ss(line);
    string firstWord;

    ss >> firstWord;

    // -----
    // MACRO START
    // -----
    if (firstWord == "MACRO") {
        insideMacro = true;
        macroName = "";
        continue;
    }

    // -----
    // MACRO HEADER
    // -----
    if (insideMacro && macroName == "") {

        macroName = firstWord;

        string arguments;
        getline(ss, arguments);

        // Remove leading spaces
        while (!arguments.empty() && arguments[0] == ' ')
            arguments.erase(0, 1);

        // Add entry to MNT
        MNT.push_back({

```

```

        mntIndex,
        macroName,
        mdtIndex
    });

    mntIndex++;

    // Parse arguments
    stringstream argStream(arguments);
    string arg;

    while (getline(argStream, arg, ',')) {

        while (!arg.empty() && arg[0] == ' ')
            arg.erase(0, 1);

        while (!arg.empty() && arg.back() == ' ')
            arg.pop_back();

        ALA.push_back({
            alalIndex,
            arg
        });

        alalIndex++;
    }

    continue;
}

// -----
// MACRO BODY
// -----
if (insideMacro) {

    // End of macro
    if (firstWord == "MEND") {

        MDT.push_back("MEND");
        mdtIndex++;

        insideMacro = false;
        macroName = "";
    }
}

```

```

        continue;
    }

    // Replace dummy arguments
    // &A -> #1
    // &B -> #2
    string modifiedLine = line;

    for (auto &entry : ALA) {

        string dummy = entry.dummyArgument;
        string positional = "#" + to_string(entry.index);

        size_t pos = modifiedLine.find(dummy);

        while (pos != string::npos) {

            modifiedLine.replace(
                pos,
                dummy.length(),
                positional
            );

            pos = modifiedLine.find(
                dummy,
                pos + positional.length()
            );
        }
    }

    MDT.push_back(modifiedLine);
    mdtIndex++;

    continue;
}

// -----
// NORMAL STATEMENT
// -----
intermediate << line << endl;
}

input.close();
intermediate.close();

```

```

// =====
// WRITE MDT
// =====

mdtFile << "INDEX\tMDT INSTRUCTION\n";
mdtFile << "-----\n";

for (int i = 0; i < MDT.size(); i++) {

    mdtFile << i + 1
        << "\t"
        << MDT[i]
        << endl;
}

mdtFile.close();

// =====
// WRITE MNT
// =====

mntFile << "INDEX\tMACRO NAME\tMDT INDEX\n";
mntFile << "-----\n";

for (auto &entry : MNT) {

    mntFile << entry.index
        << "\t"
        << entry.macroName
        << "\t\t"
        << entry.mdtIndex
        << endl;
}

mntFile.close();

// =====
// WRITE ALA
// =====

alaFile << "INDEX\tDUMMY ARGUMENT\n";
alaFile << "-----\n";

```

```

for (auto &entry : ALA) {

    alaFile << entry.index
        << "\t"
        << entry.dummyArgument
        << endl;
}

alaFile.close();

// =====
// DISPLAY OUTPUT
// =====

cout << "\n===== \n";
cout << "    INTERMEDIATE FILE \n";
cout << "===== \n";

ifstream inter("intermediate.txt");

while (getline(inter, line))
    cout << line << endl;

inter.close();

cout << "\n===== \n";
cout << "    MACRO DEFINITION TABLE (MDT) \n";
cout << "===== \n";

cout << "Index\tMDT Instruction \n";

for (int i = 0; i < MDT.size(); i++) {

    cout << i + 1
        << "\t"
        << MDT[i]
        << endl;
}

cout << "\n===== \n";
cout << "    MACRO NAME TABLE (MNT) \n";
cout << "===== \n";

cout << "Index\tMacro Name\tMDT Index \n";

```

```

for (auto &entry : MNT) {

    cout << entry.index
        << "\t"
        << entry.macroName
        << "\t\t"
        << entry.mdtIndex
        << endl;
}

cout << "\n===== \n";
cout << "    ARGUMENT LIST ARRAY (ALA)\n";
cout << "===== \n";

cout << "Index\tDummy Argument\n";

for (auto &entry : ALA) {

    cout << entry.index
        << "\t"
        << entry.dummyArgument
        << endl;
}

return 0;
}

```

input.asm-

```

START
MACRO
INCR &A,&B
LDA &A
ADD &B
STA &A
MEND
INCR X,Y
END

```

OUTPUT-

```
Enter input file name: input.asm

=====
                        INTERMEDIATE FILE
=====
START
INCR X,Y
END

=====
                        MACRO DEFINITION TABLE (MDT)
=====
Index   MDT Instruction
1       LDA #1
2       ADD #2
3       STA #1
4       MEND

=====
                        MACRO NAME TABLE (MNT)
=====
Index   Macro Name      MDT Index
1       INCR            1

=====
                        ARGUMENT LIST ARRAY (ALA)
=====
Index   Dummy Argument
1       &A
2       &B

Process returned 0 (0x0)   execution time : 2.875 s
Press any key to continue.
```